Dropout

Contents

1. Overview 2
2. Prerequisites 3
3. Overfitting in neural networks 3
4. How dropout works 3
    4.1 Switching nodes off 4
    4.2 The dropout rate 5
5. Why switching nodes off helps 6
    5.1 A smaller network in every step 6
    5.2 No node can rely on one input 6
    5.3 An ensemble, like a random forest 7
6. Dropout at prediction time 8
7. Summary 11
8. Sources 11
9. Key terms 12

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Overfitting; Regularisation; Improving a neural network.
- **Compare with:** Random forest; L1 and L2 regularisation in neural networks; Batch normalisation.

1. Overview

Key point: Dropout fights overfitting by switching off a random set of nodes at every training step, so the network trains a different, smaller **sub-network** (G-1909) each time. At prediction time every node is back.

Neural networks overfit easily (**overfitting**, G-1429), and **dropout** (G-639) is one of the most used remedies. Dropout was proposed by Nitish Srivastava, Geoffrey Hinton and colleagues (Hinton et al. 2012; Srivastava et al. 2014), and since then it has become a standard part of training networks.

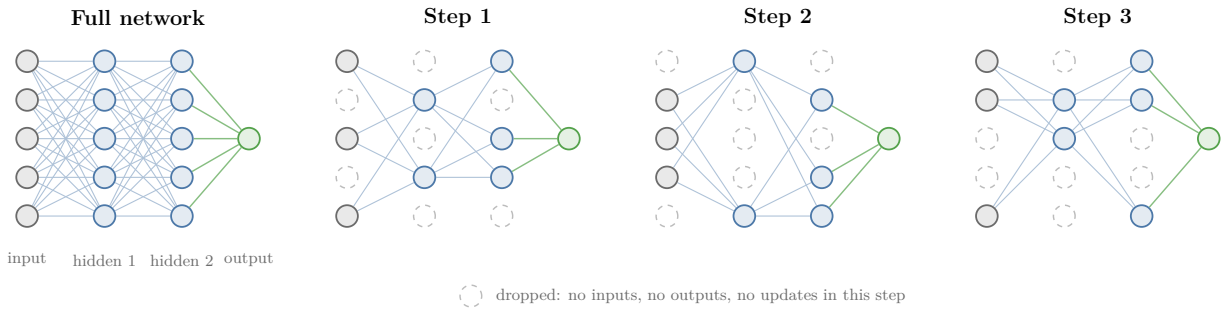


Figure 1: Dropout on a network with 5 inputs, two hidden layers of 5 nodes and 1 output. At every training step a new random set of nodes (dashed) is switched off.

Figure 1 shows the idea. The left drawing is the full network (circles are nodes, lines are connections, one column of circles per layer). The other three show the same network at three training steps. A dashed circle is a node switched off in that step, and its lines are gone. Look at which circles are dashed: the set changes from step to step. This Note explains:

- how dropout works (section 4);
- why switching nodes off helps (section 5);
- what happens at prediction time (section 6).

Code for regression and classification is in dropout for regression and dropout for classification.

2. Prerequisites

- **Overfitting** (G-1429): a model that fits the training data's noise and fails on new data.
- Column sampling and voting: how a **random forest** (G-1611) builds many different trees and combines them.
- Fixing overfitting: where **dropout** (G-639) sits among the fixes for a neural network.

3. Overfitting in neural networks

> **Key point:** Many layers of many fully connected nodes can capture every small pattern of the training data, noise included. Five fixes: more data, a simpler network, early stopping, regularisation, **dropout** (G-639).

Overfitting is learning the training data too closely, noise included, so that the model does well on the training data and poorly on new data (see **overfitting** (G-1429)). In a classification problem, an overfitted model has a wiggly **decision boundary** (G-555; the line or curve separating the classes) that wraps around every training point; a good model has a smoother decision boundary that captures the real pattern.

Neural networks are prone to overfitting, for three reasons:

- they have several layers, each with many nodes;
- the layers are all fully connected (**dense layers**, G-583);
- they are trained for many **epochs** (G-696).

Such a complex **architecture** (G-209) can capture tiny patterns that only exist in the training data.

The possible fixes:

1. **Add more data.** The more varied data the network sees, the better it generalises (**generalisation**, G-838).
2. **Reduce the complexity** of the network: for example 7 **hidden layers** (G-890) instead of 10, or 64 nodes per layer instead of 128.
3. **Early stopping** (G-656): stop training where overfitting starts (see early stopping).
4. **Regularisation** (G-1659), L1 or L2, as in Ridge and Lasso (see the **penalty term** (G-1476)).
5. **Dropout**, the subject of this Note.

4. How dropout works

> **Key point:** At each training step, every node of the input and hidden layers is switched off with probability p, the dropout rate. The **output layer** (G-1424) is never dropped.

4.1 Switching nodes off

> **Key point:** A dropped node has no connections in that step: it sends nothing, receives nothing, and its weights are not updated.

Take a binary classification problem with 5 **features** (G-772; input variables, one column of the data table each). The network (Figure 1, left) has:

- an **input layer** (G-952) of 5 nodes;
- two **hidden layers** (G-890) of 5 nodes each;
- an **output layer** (G-1424) of 1 node.

With **dropout**, before each training step we randomly switch off some nodes of the input layer and of the hidden layers. A switched-off node is cut from the network for that step: its weights and bias play no part in the prediction, and **backpropagation** (G-247) does not update them. In Figure 1, step 1 drops 2 input nodes, 3 nodes of the first hidden layer and 2 of the second. Step 2 draws a new random set: some of the dropped nodes come back and others go.

So each step trains a different network on the data. Every one of them is a smaller **sub-network** (G-1909) of the full network, using a subset of its nodes and the same shared weights.

The output layer is never switched off, because every sub-network must still make a full prediction: in training that prediction is scored against the true label, and at prediction time the predictions of all the sub-networks are averaged. So **dropout** (G-639) removes only nodes that are not outputs (Goodfellow, Bengio and Courville, *Deep Learning*, §7.12; Srivastava et al. 2014, §1).

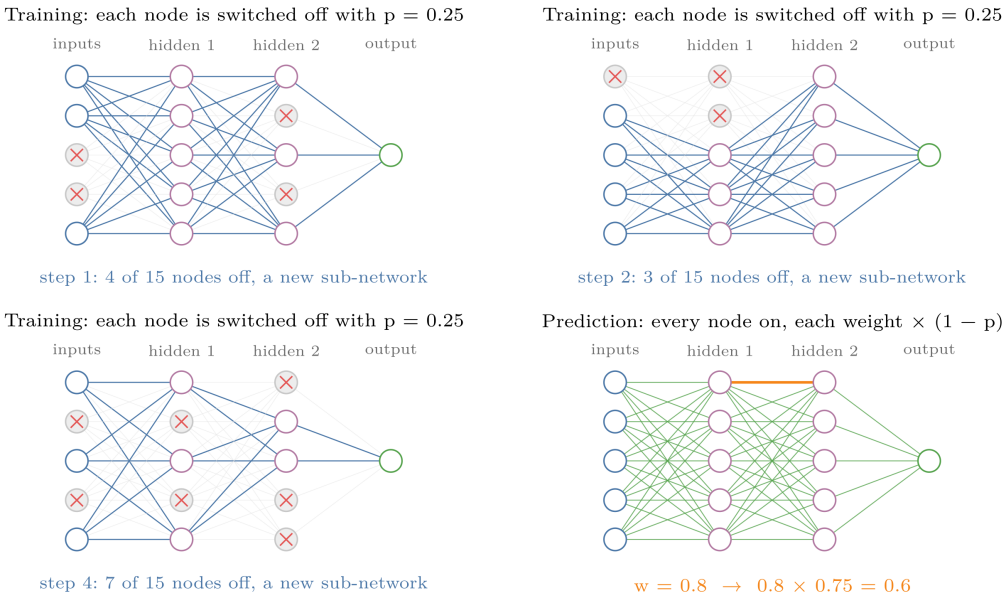


Figure 2: Dropout with p = 0.25 on the network of Figure 1, over four training steps. Each step draws a new random set of switched-off nodes (red crosses), and their connections go grey. At prediction every node is back, and each weight is multiplied by $1 - p = 0.75$

In Figure 2, watch the crosses jump between steps: 4, 3, 1 and then 7 of the 15 droppable nodes are off, because every node is dropped independently (section 4.2). The last frame previews prediction time (section 6).

> **Extra:** The random on/off pattern (the **mask**) is redrawn for every forward pass, which in Keras means for every **mini-batch** (G-263; and each **observation** (G-1374), one record, in the batch gets its own mask), not once per **epoch** (G-696) (Keras docs, `Dropout`). With 10 epochs of 100 mini-batches, the network trains on at least 1,000 different sub-networks (more, since each observation has its own mask), not 10.

4.2 The dropout rate

> **Key point:** p is the probability of dropping each node; it can differ from layer to layer.

The **dropout rate** (G-638) p is the fraction of a layer's nodes to drop. With $p = 0.5$, each node of that layer is switched off with probability 0.5, so on average half the layer is gone in each step. With $p = 0.25$, each node is switched off with probability 0.25, so on average one node in four is gone; with a layer of 100 nodes that is about 25 nodes. Each layer can have its own rate; typical values come in choosing the dropout rate.

5. Why switching nodes off helps

> **Key point:** Three views of the same effect: a smaller network each step; no node can rely on one input; and many sub-networks vote, like a **random forest** (G-1611).

Breaking the network on purpose sounds like it should make it worse. In practice it makes it better: in the original experiments, **dropout** (G-639) improved even strong networks by about 1 to 2 percentage points (Srivastava et al. 2014, §6), which is a big jump when the accuracy is already 95%.

5.1 A smaller network in every step

> **Key point:** Fewer active nodes means fewer connections, so less capacity to fit tiny details.

There are two ways to reduce **overfitting** (G-1429) in a network: reduce the number of nodes, or stop nodes from focusing on one particular pattern. Dropout does both.

Every **sub-network** (G-1909) is smaller than the full one. Fewer nodes mean fewer connections and so fewer possibilities to fit small details of the training data. Each step, the network simply does not have the resources to capture very fine patterns.

5.2 No node can rely on one input

> **Key point:** Since any input may vanish in the next step, a node spreads its weights over all its inputs instead of leaning on one.

Take one node of the second **hidden layer** (G-890), receiving input from 4 nodes of the first. Depending on the data, it may start to rely mostly on one of them: one large weight and three small ones. The node is then focused on one particular pattern.

With **dropout** (G-639), that favourite input is missing in some steps. The node cannot know whether it will be there in the next step, so it has to make use of the other three too. Its attention is divided over all its inputs, the weights become more balanced, and the network ignores small, isolated patterns in favour of the overall pattern of the data.

A company analogy helps. Suppose every morning, a random half of the employees is told to stay at home. Each employee can no longer depend on one particular colleague (the only tester, say) and learns to work with several. The company as a whole becomes more resilient: insensitive to the absence of any one person. For a company this may cost productivity; for a neural network it works.

5.3 An ensemble, like a random forest

> **Key point:** Dropout trains a huge number of **sub-networks** (G-1909) that share their weights; prediction combines them, as a random forest combines its trees.

A **random forest** (G-1611) trains many **decision trees** (G-561), each on a random sample of the features (**column sampling**, G-413; see random forest by hand), and lets them vote. The trees differ a little from each other, and the vote of the **ensemble** (G-690) overfits much less than any one tree.

Dropout (G-639) does the same with networks. Each training step trains a different sub-network on the data; at prediction time their knowledge is combined. And the number of possible sub-networks is enormous:

1. **In words:** every droppable node is either present or absent, 2 options each, so n droppable nodes give 2^n sub-networks.
2. **Formula:**
$$\text{number of sub-networks} = 2^n$$
3. **Example:** a network with 4 inputs, 4 hidden nodes and 1 output has $n = 8$ droppable nodes (the output is never dropped): $2^8 = 256$ sub-networks. The network of Figure 1 has $n = 15$: $2^{15} = 32{,}768$.

With so many possibilities, the same sub-network is very unlikely to come up twice in training. Training for 100 steps trains 100 different networks, close relatives because they share weights, and the final network behaves like their ensemble (Srivastava et al. 2014, §1). The ensemble view is why dropout is often compared with a random forest.

Figure 3 checks the ensemble view on a trained network. A network with two **hidden layers** (G-890) of 128 ReLU nodes (ReLU outputs $\max(0, z)$ for an input z) and $p = 0.5$ after each was trained on 200 points of `make_moons` (two interleaved half-moons, the data of regularisation in Keras). Then we pick sub-networks: for each one, a single random set of nodes is switched off, and the same sub-network classifies every point of the plane.

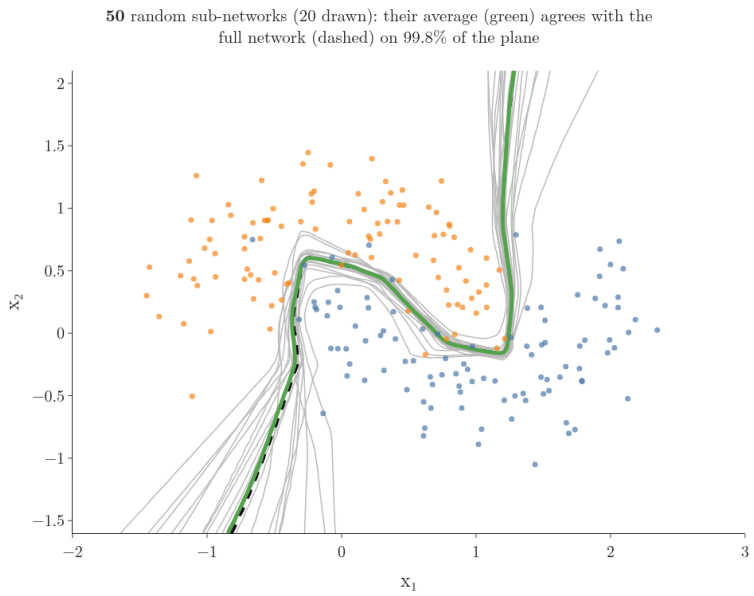


Figure 3: Sub-networks of one trained dropout network. Grey: the decision boundaries of single random sub-networks. Green: the boundary of their average output. Dashed black: the full network with every node, as used at prediction.

In Figure 3, each line is a decision boundary: the places where a network's output is 0.5, so the predicted class switches from one side to the other. No background is coloured here, and the lines are not contours of a loss surface.

- **Single sub-networks (grey)** all follow the two moons where the training points are, but disagree far from them, for example at the bottom left and top right.
- **Their average (green)** settles as more sub-networks are added. With 20 or more, it agrees with the full network on 99.8 percent of the plane.
- **The full network (dashed)** therefore behaves like the vote of many sub-networks, without running any of them separately. Section 6 explains the scaling that makes this work.

6. Dropout at prediction time

> **Key point: Dropout** (G-639) is applied only during training. At prediction every node is present, and each weight is multiplied by $1 - p$ to make up for it.

During testing and prediction no node is dropped: every node and every connection is active. Using every node creates a mismatch. During training, a node was present only part of the time, so the nodes after it got used to receiving its signal only that often. At prediction it is always present.

The fix is to scale the weights down by the probability that the node was present.

1. **In words:** a node dropped with probability p was present in a fraction $1 - p$ of the

training steps; its outgoing weights are multiplied by $1 - p$ at prediction.

2. **Formula:**

$$w_{\text{test}} = w \times (1 - p)$$

3. **Example:** with $p = 0.25$, a node is present in about 75 of 100 training steps. A weight $w = 0.8$ learned in training becomes

$$w_{\text{test}} = 0.8 \times (1 - 0.25) = 0.6$$

On average, the next node then receives the same signal at prediction as it did during training.

Figure 4 shows the two ways (left column: training, right column: prediction). In the top row, the weights are scaled down by $1 - p$ at prediction (the version derived here). In the bottom row, the kept outputs are scaled up during training and prediction needs no change. Both give the next layer the same average signal. We never have to do it by hand: Keras handles it behind the scenes.

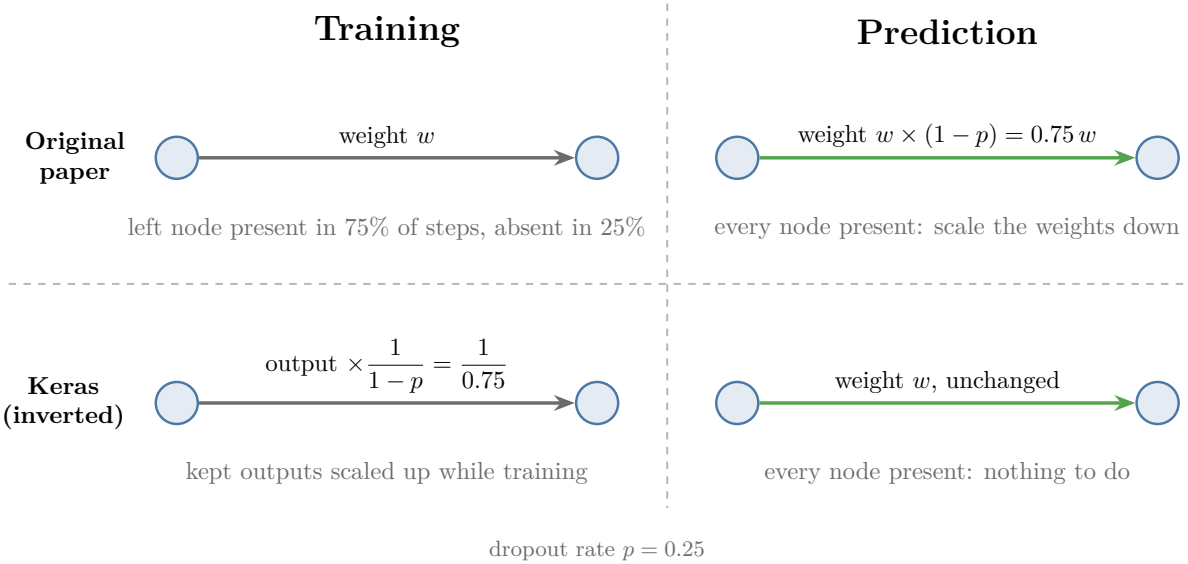


Figure 4: Two equivalent ways to handle dropout. Top: the original paper scales the weights down at prediction. Bottom: Keras scales the kept outputs up during training, so nothing changes at prediction.

Extra: Keras uses **inverted dropout** (G-970; Figure 4, bottom). During training it divides the output of every kept node by $1 - p$; at prediction it does nothing (Keras docs, `Dropout`). Both versions give the next layer the same average input.

Python: A `Dropout(0.25)` layer on a row of eight 1s.

```
import keras, numpy as np
keras.utils.set_random_seed(0)
```

```
drop = keras.layers.Dropout(0.25)
x = np.ones((1, 8), "float32")
drop(x, training=True)   # [1.333 1.333 1.333 0 1.333 0 1.333 1.333]
drop(x, training=False)  # [1 1 1 1 1 1 1 1]
```

In training, two of the eight values are dropped and the others become $1/0.75$, which is 1.333. `fit` passes `training=True` automatically; `predict` and `evaluate` pass `training=False`.

7. Summary

	During training	During prediction
Nodes	each input and hidden node dropped with probability p	all present
Network	a different random sub-network every step	the full network
Weights (original paper)	w	$w \times (1-p)$
Keras (inverted dropout)	kept outputs divided by $1-p$	unchanged

- Dropout switches off random nodes at each training step; the **output layer** (G-1424) is never dropped, because every sub-network must still give a full prediction to score and to average.
- A smaller network each step, and no node can rely on a single input: both reduce **overfitting** (G-1429).
- With n droppable nodes there are 2^n sub-networks; dropout trains an ensemble of them, like a **random forest** (G-1611), and the full network at prediction acts like their average.
- At prediction every node is back, with its weights scaled by $1-p$ (Keras does the equivalent automatically), so the next node receives the same average signal as during training.

8. Sources

Built from

- CampusX, "Dropout Layer in Deep Learning | Dropouts in ANN | End to End Deep Learning", YouTube, https://www.youtube.com/watch?v=gyTlcHVeBjM

Other references

- Hinton, Srivastava, Krizhevsky, Sutskever and Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors", arXiv:1207.0580, 2012.
- Srivastava, Hinton, Krizhevsky, Sutskever and Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting", *JMLR*, 2014: §1 (units dropped are hidden and visible units; 2^n thinned networks, approximate averaging), §6 (results).
- Goodfellow, I., Bengio, Y. and Courville, A., *Deep Learning*, MIT Press, 2016, deeplearningbook.org, §7.12 "Dropout" (the ensemble of all sub-networks formed by removing non-output units, Figure 7.6; each sub-network gives its own prediction, and the ensemble averages them).
- Keras documentation, `keras.layers.Dropout` (applied at each training step; kept inputs scaled by $1/(1-\text{rate})$).

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sub-network (G-1909)	The smaller network left after dropout switches some nodes off; it shares the full network's weights, and each training step trains a different one.
Dropout (G-639)	Switching off a random set of input and hidden nodes at every training step, to reduce overfitting.
Dropout rate (p) (G-638)	The probability that each node of a layer is switched off in a training step.
Inverted dropout (G-970)	Dropout that scales the kept outputs up by $1/(1-p)$ during training, so prediction needs no change; what Keras does.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Random forest (G-1611)	Bagging with decision trees as the base models: many trees, each trained on a random sample of the rows with a random choice of features at every split, vote (classification) or are averaged (regression), which lowers the variance of a single tree.
Decision boundary (G-555)	A line or curve that separates the classes in classification.
Dense (fully connected) layer (G-583)	A layer whose every node receives the output of every node in the layer before.
Epoch (G-696)	One full update of the parameters using the whole training set.
Architecture (of a network) (G-209)	How a network's nodes are connected: how many, of what kind, and which connections.
Generalisation (generalise) (G-838)	How well a model performs on new data it was not trained on.
Hidden layer (G-890)	A layer of neurons between the input and output layers; it works on the output of the layer before and passes what it finds on, so the network can build up more complex patterns.
Early stopping (G-656)	Stopping training when the score on held-out data is best, before full convergence; this keeps coefficients small, so it also acts as regularisation.

Term	Meaning
Regularisation (G-1659)	Adding a penalty for large coefficients to the loss, so the model overfits less (lower variance).
Penalty term (G-1476)	The extra term added to the cost to discourage large weights.
Output layer (G-1424)	The last layer, which gives the prediction.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Input layer (G-952)	The first layer of a neural network, with one node per input column; it takes in the data and passes the values on without calculating anything.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Batch (mini-batch) (G-263)	A small group of training observations used for one update; Keras uses 32 by default.
Observation (G-1374)	One record of a dataset: one row of the data table.
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Column sampling (feature sampling) (G-413)	Training each model of an ensemble (each base model) on a random subset of the columns.
Ensemble (G-690)	Several trained models whose predictions are combined.